Hamburg University of Technology
Institute of Mathematics
Chair Numerical Mathematics

Jens-Peter M. Zemke
Finn Sommer
Lars Stietz

Advanced Machine Learning
Winter Semester 2025/2026
Exercise Sheet 4

Week 5, 10.11.2025 – 14.11.2025

Remark: In order to use our library it has to be registered with `pip` for each exercise sheet. You can use the script `uninstall_amllib.py` to make sure the library version from the last exercise sheet is deleted. Then use `install_amllib.py` to register the library version for this exercise sheet. The script `install_amllib` registers the library as editable library, hence all changes are directly accessible. The script `uninstall_amllib.py` reverts the installation.

Homework:

Exercise 1: (2+2 points) level ●●○ relevance ●●●
In multiclass classification one uses as activation function in the last layer the so-called $\mathsf{SoftMax}$ function,

$$\mathsf{SoftMax} : \mathbb{R}^n \to (0,1)^n, \quad \mathbf{p} := \mathsf{SoftMax}(\mathbf{x}) := \frac{e^{\mathbf{x}}}{\mathbf{e}^\mathsf{T} e^{\mathbf{x}}} = \begin{pmatrix} e^{x_1} \\ \vdots \\ e^{x_n} \end{pmatrix} / \Big(\sum_{j=1}^{n} e^{x_j} \Big).$$

This function returns a probability distribution, the sum of all entries is one. It is commonly used together with the *Cross Entropy Loss* function

$$H(\mathbf{y}, \mathbf{p}) := -\sum_{j=1}^{n} y_j \ln(p_j) \in (0, \infty)$$

as cost function. In the training, the vector $\mathbf{y}$ will be a standard unit vector corresponding to the one-hot encoded class, $\mathbf{p}$ will be the outcome of the $\mathsf{SoftMax}$-layer. This is known as categorical cross entropy loss. Compute the elements of the derivative/gradient of

a) the $\mathsf{SoftMax}$ function $\mathbf{p} = \mathsf{SoftMax}(\mathbf{x})$ with respect to $\mathbf{x}$,
b) the Cross Entropy Loss combined with the $\mathsf{SoftMax}$ layer, i.e., $H(\mathbf{y}, \mathsf{SoftMax}(\mathbf{x}))$, with respect to $\mathbf{x}$.

How do you initialize backpropagation in this case?

Exercise 2: (4 points) level ●○○ relevance ●●○ required for exercises 4 and 5
The initial choice of the weights effects the convergence behavior of the SGD method. Add different weight initialization schemes to the `initializers` package, as treated in the lecture:

a) A class `RandnAverage` that provides the method `wfun(self, m, n)` that returns an $m \times n$ random matrix with normally distributed entries and variance $2/(m+n)$. Hint: Use `numpy.random.randn` and multiply the result by the square root of the desired variance.

b) A class `RandAverage` that provides the method `wfun(self, m, n)` that returns an $m \times n$ random matrix with uniformly distributed entries on the interval $[r, r]$, where the radius is given by $r := \sqrt{6/(m+n)}$. Hint: Use `numpy.random.rand`. The method `rand` generates random numpy arrays with entries uniformly distributed in the interval $[0, 1]$. Shift and scale the results in an appropriate way.

c) A class `RandnOrthonormal` that provides the method `wfun(self, m, n)` that returns an $m \times n$ random orthonormal matrix $\mathbf{W}$, i.e.,

$$\mathbf{W}^\mathsf{T}\mathbf{W} = \mathbf{I}_n, \quad \text{or} \quad \mathbf{W}\mathbf{W}^\mathsf{T} = \mathbf{I}_m$$

holds if $n \leqslant m$ or $m \leqslant n$, respectively. Hint: Use `numpy.linalg.svd` or `numpy.linalg.qr` as sketched in the lecture to obtain a random orthonormal matrix from a random matrix.

All initializers should inherit from the `KernelInitializer` base class provided in the skeleton in the file `initializer.py`.

Exercise 3: (2+2 points) level •○○ relevance •••

In this exercise we consider part of the proof of the backpropagation for the computation of gradients. Let

$$\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}, \quad \mathbf{z}, \mathbf{b} \in \mathbb{R}^m, \quad \mathbf{a} \in \mathbb{R}^n, \quad \mathbf{W} \in \mathbb{R}^{m \times n}$$

denote the affine mapping in one of the layers. Let C be a cost function and let

$$\boldsymbol{\delta} = \frac{\partial C}{\partial \mathbf{z}} = \nabla_{\mathbf{z}} C \in \mathbb{R}^m$$

be given. Let the 3-tensor[1] $\mathbf{T} \in \mathbb{R}^{m \times m \times n}$ be defined by elements

$$t_{i,j}^{(\ell)} := \frac{\partial z_\ell}{\partial w_{i,j}}$$

for $\ell = 1, \ldots, m$, $i = 1, \ldots, m$, $j = 1, \ldots, n$.

a) Compute for $\ell = 1, \ldots, m$ the 2-tensor slices

$$\mathbf{T}^{(\ell)} := \begin{pmatrix} t_{1,1}^{(\ell)} & \cdots & t_{1,n}^{(\ell)} \\ \vdots & \ddots & \vdots \\ t_{m,1}^{(\ell)} & \cdots & t_{m,n}^{(\ell)} \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

b) Show that the application of the 3-tensor $\mathbf{T}$ to the 1-tensor $\boldsymbol{\delta}$ gives the same 2-tensor as obtained using the elementwise chain rule,

$$\mathbf{T}\boldsymbol{\delta} := \sum_{\ell=1}^{m} \mathbf{T}^{(\ell)} \mathbf{e}_\ell^\mathsf{T} \boldsymbol{\delta} = \boldsymbol{\delta}\mathbf{a}^\mathsf{T} = \frac{\partial C}{\partial \mathbf{W}} \in \mathbb{R}^{m \times n}$$

Exercises (in class):

Exercise 4: (4 points) level ••○ relevance •••

Implement a sequential network class like in TensorFlow (`tf.keras.Sequential`) and PyTorch (`torch.nn.Sequential`).

[1] A 3-tensor is a natural generalization of a scalar (0-tensor), column vector (1-tensor), and matrix (2-tensor) to an additional dimension. Think of it as a cube of numbers, or as matrices stacked on top of each other towards the reader. Here, it makes sense to think of it as a row vector of matrices.

a) Complete the implementation of a dense connected layer in `layers/dense.py`. Add
   (i) the feedforward evaluation of the layer in the method `feedforward`,
   (ii) the backpropagation of a dense connected layer in the method `backprop`
   (iii) the update[2] of the weights and the bias in the method `update`.

   For this implementation assume the input data in a transposed format, like in TensorFlow and PyTorch due to the way of storing vectors in Python. This results in a different order of the weight matrix multiplication. In the `FeedforwardNet` implementation we evaluated dense layers with

$$\mathbf{z}_i = \mathbf{W}_i\mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{z}_i, \mathbf{b}_i \in \mathbb{R}^m, \quad \mathbf{a}_i \in \mathbb{R}^n, \quad \mathbf{W}_i \in \mathbb{R}^{m\times n}.$$

   All vectors were *column* vectors and we worked with gradients. Now the inputs are *row* vectors and we work with derivatives, i.e., the evaluation changes to

$$\mathbf{z}_i = \mathbf{a}_i\mathbf{W}_i + \mathbf{b}_i, \quad \mathbf{z}_i, \mathbf{b}_i \in \mathbb{R}^{1\times m}, \quad \mathbf{a}_i \in \mathbb{R}^{1\times n}, \quad \mathbf{W}_i \in \mathbb{R}^{n\times m},$$

   where the new weight matrix is the transpose of the old one.
   The backpropagation should use a loss function, which is set in the `__init__` method. Loss functions are implemented in the `losses` sub-package and provide two methods. A `__call__` method responsible for the computation of the loss and a `backprop` method responsible for computing the derivative of the loss with respect to the networks output.

b) Complete the skeleton in `networks/sequential.py`. Implement
   (i) the feedforward evaluation of the network in the method `feedforward`,
   (ii) the backpropagation through the network in the method `backprop`,
   (iii) the training of the network in the method `train`.

c) Test your implementation with the MNIST dataset and the network structure from exercise 4 on exercise sheet 2. To load and preprocess the MNIST dataset, the package `utils` provides the module `mnist`. This module provides the functions
   - `load_data()` to load the dataset (the pixel values are scaled to $[0, 1]$),
   - `encode_labels(labels)` to one-hot encode the labels,
   - `flatten_input(input)` to flatten the input from shape $(28, 28)$ to shape $(784,)$.

Exercise 5: (4 points) level ●○○ relevance ●●○

Use the skeleton provided in `exercise5_skel.py` to set up two feedforward neural networks with one hidden layer, one network for the MNIST dataset and one for the so-called fashion MNIST dataset. Compare the loss of both networks throughout the training for different parameter choices. What do you observe?

[2] Use `p -= a*dp` instead of `p = p - a*dp` to update a parameter `p` within the update method since the update is discarded otherwise due to the argument handling in Python.